TSP

Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)

Case Study: Design & Analysis of Algorithm

Topic: 82. Traveling Sales Route Planning

Guided BY-
Dr. Sarika Deokate

BY-
Parth Patil 123B1B213

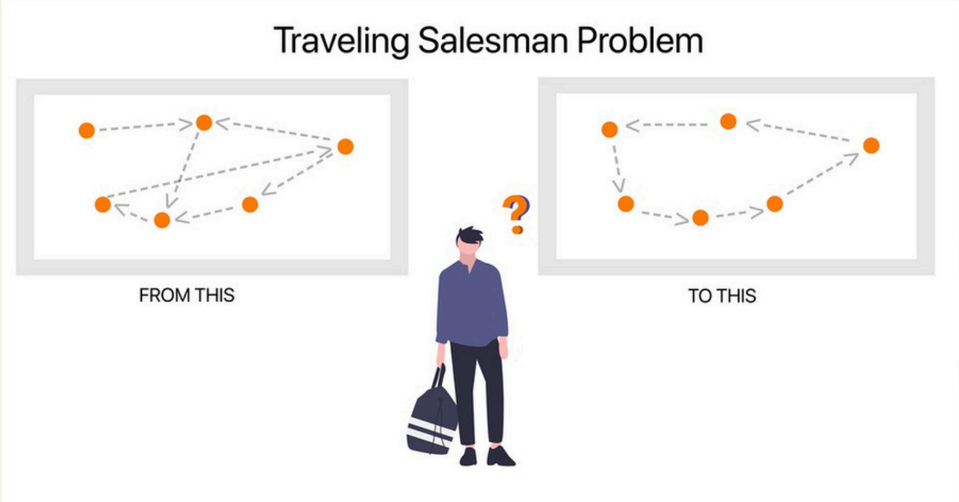
Problem Introduction

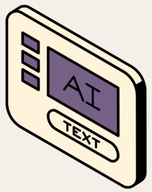
Travelling Salesman Problem (TSP)

- Visit all cities exactly once
- Return to starting city
- Minimize total travel distance

Problem Definition

- Given: Distance between every pair of cities
- Find: Shortest possible route covering all cities
- Type: Optimization problem (NP-Hard)





framework

Possible Approaches

Possible Approaches

- Brute Force
- Greedy (Nearest Neighbor)
- Dynamic Programming
- Branch and Bound
- Held Carp Algorithm
- **Selected: Branch and Bound**
- Gives optimal solution
- Uses pruning to reduce search More efficient than brute force

Algorithm Steps + Pseudocode

Algorithm Steps (Branch & Bound)

Start from initial city
Initialize:
- bestCost = INFINITY
- Priority Queue (min-heap)

Create root node:
- path = [start]
- cost = 0
- bound = lower bound

Repeat:
- Select node with minimum bound

If all cities visited:
  - Add cost to return to start
  - Update bestCost

Else:
  - Generate child nodes
  - Compute bound
  - Add if bound < bestCost

Stop when queue is empty

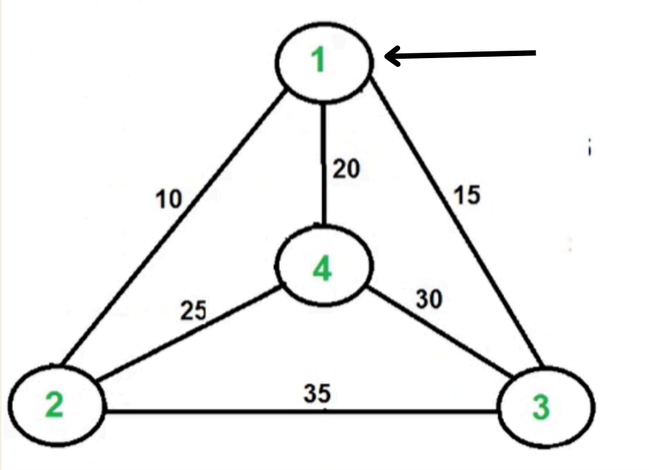
Pseudocode

```
Initialize bestCost = INIFNITY
Create root node and push to PQ
While PQ not empty:
   node = remove min bound
   If bound ≥ bestCost:
      continue
   If all cities visited:
      cost = costSoFar + return edge
      update bestCost
   Else:
      For each unvisited city:
         create child
         compute bound
         If bound < bestCost:
            push to PQ
Return best path
```

Example

State Space Tree Explanation

- Root node → starting city
- Each level → selection of next city
  Branches → all possible choices
- Leaf nodes → complete tours (return to city 1)
- Total nodes ≈ permutations of cities
- Pruning:If partial path cost is already high → stop expanding

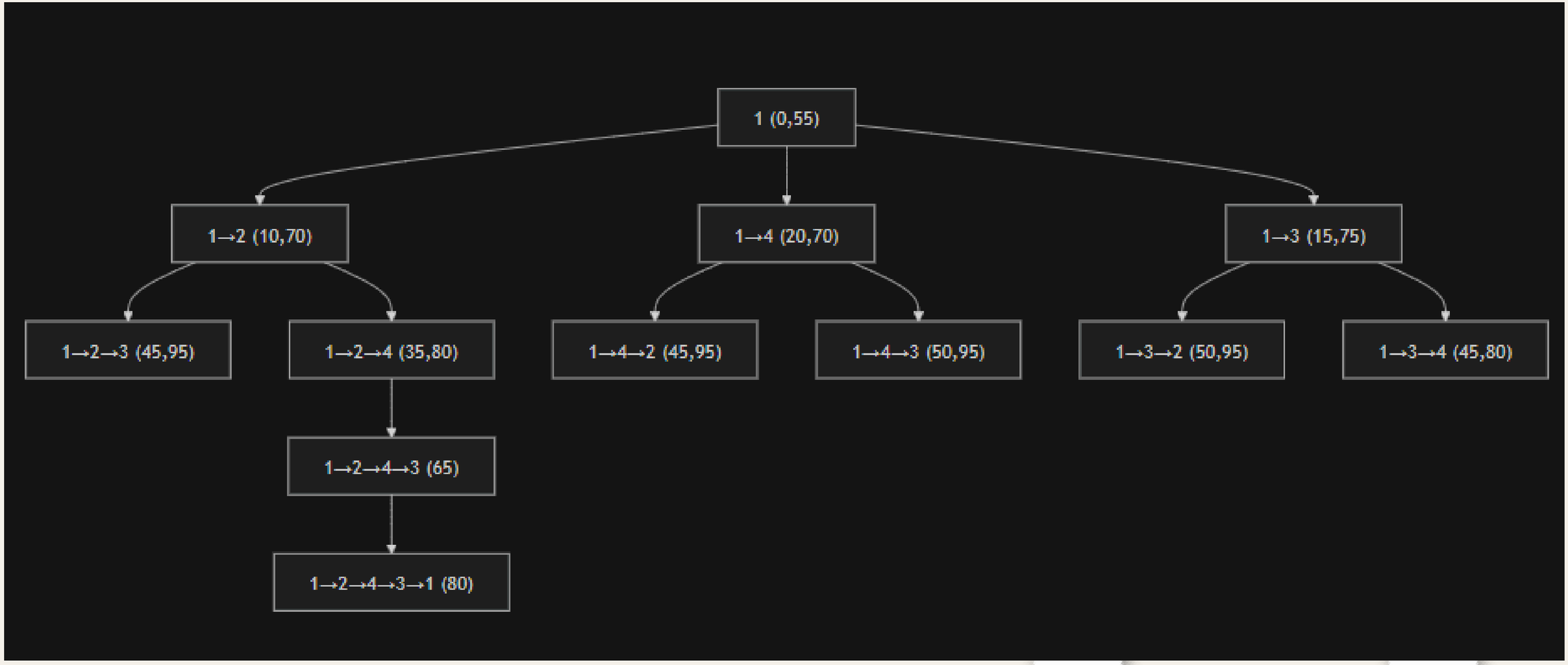


Distance table:

	C1	C2	C3	C4
C1	0	10	15	20
C2	10	0	35	25
C3	15	35	0	30
C4	20	25	30	0

```
Final tour building steps:
city 1[0] -> city 2[10]
city 1[0] -> city 2[10] -> city 4[35]
city 1[0] -> city 2[10] -> city 4[35] -> city 3[65]
city 1[0] -> city 2[10] -> city 4[35] -> city 3[65] -> city 1[80]
```

State Space Tree



Time Complexity Analysis

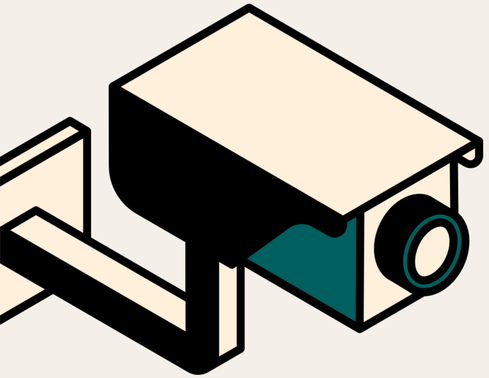
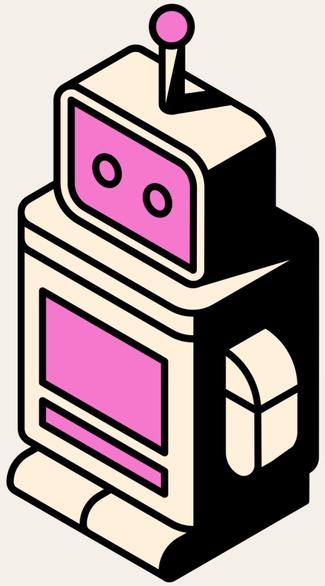
State Space Tree structure
At each level of the tree, we choose the next unvisited city:
- Level 0 (root): 1 node — start city fixed
- Level 1: up to (N-1) children
- Level 2: up to (N-2) children each
- Level N-1: 1 child each (last city), then return to start

So the total nodes without pruning = 1 + (N-1) + (N-1)(N-2) + ... = O(N!)

Why B&B beats Brute Force
Brute force always runs in O(N!) — it checks every permutation.
B&B prunes any branch where `current_cost ≥ best_known_tour`, so large portions of the tree are never visited.

For our 4-city example
Brute Force: 24 possible tours (4!)
Branch & Bound: only a small subset explored (pruned search)

Space complexity
O(N) — only the current path is stored on the recursion stack (depth = N).



Thank You